```
using System.Net.Http.Headers;
using System.IO;
using System.Runtime.CompilerServices;
namespace WheelOfFortune
{
    internal class Program
    {

        //userChoice in main menu
        public static int userChoice;


        //struct student
        public struct Student
        {
            public string firstName;
            public string lastName;
            public string interest;
            public int score;
        }

        //create streamwriter and streamreader
        public static StreamWriter sw;
        public static StreamReader sr;

        public static void Main(string[] args)
        {
            //initializing variables
            Student[] student = new Student[23];
            Student[] finalists = new Student[10];
            Student selectedPlayer = new Student();

            //call the text file
            LoadStudents(student);

            //runs the program while userChoice is not the exit code
            do
            {
               selectedPlayer = Output(student, finalists,
                 selectedPlayer);
            } while (userChoice != 0);
        }

        //get the students from the text file to avoid dependencies of
          each task
        public static void LoadStudents(Student[] student)
        {
            int i = 0;
            sr = new StreamReader(@"../../../wheelOfFortune.txt");
            while(!sr.EndOfStream && i < student.Length)
            {
                student[i].firstName = sr.ReadLine();
                student[i].lastName = sr.ReadLine();
```

```
52                  student[i].interest = sr.ReadLine();
53                  student[i].score = int.Parse(sr.ReadLine());
54                  i++;
55              }
56              sr.Close();
57          }
58
59          //taskone to display students ascending by last name
60          public static void TaskOne(Student[] student, Student
              selectedPlayer)
61          {
62              Console.Write("First Name".PadRight(25));
63              Console.Write("Last Name".PadRight(25));
64              Console.Write("Interest".PadRight(50));
65              Console.WriteLine("Score".PadRight(25));
66              Console.WriteLine();
67
68              //display students
69              for(int i = 0; i < student.Length; i++)
70              {
71                  for(int j = 0; j < i; j++)
72                  {
73                      //bubble sorting strings
74                      if (student[j].lastName.CompareTo(student
                          [i].lastName) > 0)
75                      {
76                          Student temp = student[i];
77                          student[i] = student[j];
78                          student[j] = temp;
79                      }
80                  }
81
82              }
83
84              for(int i = 0; i < student.Length; i++)
85              {
86                  Console.Write($"{student[i].firstName}".PadRight(25));
87                  Console.Write($"{student[i].lastName}".PadRight(25));
88                  Console.Write($"{student[i].interest}".PadRight(50));
89                  Console.WriteLine($"{student[i].score}".PadRight(25));
90              }
91          }
92
93          //task to find a specific student by last name
94          public static void TaskTwo(Student[] student)
95          {
96              int choice;
97              do
98              {
99                  Console.Clear();
100
101                 //created a mini menu if the user wants to update a
                      student interest
```

```
102                 Console.WriteLine("1 - Find a student by last name");
103                 Console.WriteLine("0 - Go back");
104                 Console.Write("Enter your choice: ");
105                 choice = int.Parse(Console.ReadLine());
106                 switch (choice)
107                 {
108                     case 1:
109                         //clears the screen for UI purposes
110                         Console.Clear();
111                         int studentFoundNumber = 0;
112                         bool studentFound = false;
113                         do
114                         {
115
116                             Console.Write("Enter a student last name:
                                ");
117                             string lastNameSearch = Console.ReadLine();
118                             for (int i = 0; i < student.Length; i++)
119                             {
120                                 //matched student name and userinput
                                    in lowercase accurately
121                                 if (lastNameSearch.ToLower() == student
                                    [i].lastName.ToLower())
122                                 {
123                                     studentFound = true;
124                                     studentFoundNumber = i;
125                                 }
126                             }
127
128                             //condition if student was find and update
                                their interest
129                             if (studentFound == true)
130                             {
131                                 Console.WriteLine("Student found!");
132                                 Console.WriteLine($"{student
                                    [studentFoundNumber].firstName}
                                    {student
                                    [studentFoundNumber].lastName}");
133                                 Console.Write("Update the interest: ");
134                                 string interest = Console.ReadLine();
135                                 student[studentFoundNumber].interest =
                                    interest;
136                                 sw = new StreamWriter(@"../../../
                                    wheelOfFortune.txt");
137                                 Console.WriteLine("List updated! Check
                                    the text file.");
138                                 for (int i = 0; i < student.Length; i+
                                    +)
139                                 {
140                                     sw.WriteLine(student[i].firstName);
141                                     sw.WriteLine(student[i].lastName);
142                                     sw.WriteLine(student[i].interest);
143                                     sw.WriteLine(student[i].score);
```

```
144                                 }
145                                 sw.Close();
146                                 Console.ReadLine();
147                             }
148                             else
149                             {
150                                 Console.WriteLine("Student not found!
                                      Try again!");
151                             }
152                         } while (studentFound == false);
153 
154 
155                         break;
156                     case 0:
157                         Console.WriteLine("Returning to main menu.");
158                         break;
159                     default:
160                         Console.WriteLine("Invalid input! Please try
                                  again!");
161                         Console.ReadLine();
162                         break;
163                 }
164             } while (choice != 0);
165 
166         }
167 
168         //method to generate 10 student finalist to play the wheel of
              fortune
169         public static void TaskThree(Student[] student, Student[]
              finalists)
170         {
171             //initialize variables
172             Random rand = new Random();
173             int[] arrayRandom = new int[finalists.Length];
174             int randomGenerator;
175 
176             Console.WriteLine("Generating 10 finalists:");
177             Console.WriteLine();
178             Console.Write("First Name".PadRight(25));
179             Console.Write("Last Name".PadRight(25));
180             Console.Write("Interest".PadRight(50));
181             Console.WriteLine("Score".PadRight(25));
182             Console.WriteLine();
183 
184             //outer loop to generate 10 students
185             for (int i = 0; i < finalists.Length; i++)
186             {
187                 //random generator to find the student according to
                      the struct
188                 randomGenerator = rand.Next(0, student.Length-1);
189 
190                 //inner loop that will start at 0 if a name has been
                      duplicated
```

```
191                 for(int k = 0; k < i; k++)
192                 {
193                     //condition if duplication occurs
194                     while (arrayRandom[k] == randomGenerator)
195                     {
196                         randomGenerator = rand.Next(0, student.Length);
197                         //k will be assigned 0 and the the loop again
                              to find if duplication happened
198                         k = 0;
199                     }
200                 }
201 
202                 arrayRandom[i] = randomGenerator;
203                 finalists[i] = student[randomGenerator];
204             }
205 
206             //display 10 finalists
207             for(int i = 0; i < finalists.Length; i++)
208             {
209                 Console.Write($"{finalists[i].firstName}".PadRight
                      (25));
210                 Console.Write($"{finalists[i].lastName}".PadRight(25));
211                 Console.Write($"{finalists[i].interest}".PadRight(50));
212                 Console.WriteLine($"{finalists[i].score}".PadRight
                      (25));
213             }
214         }
215 
216         //method to select one student among the 10 finalists
217         //this method will not work if finalists are not generated
218         public static Student TaskFour(Student[] finalists, Student
              selectedPlayer)
219         {
220             Random rand = new Random();
221             int randomGenerator;
222             randomGenerator = rand.Next(0, finalists.Length);
223             Console.WriteLine("Congratulations! You have selected");
224 
225             selectedPlayer = finalists[randomGenerator];
226             Console.WriteLine($"{selectedPlayer.firstName}
                  {selectedPlayer.lastName} {selectedPlayer.score}");
227             Console.WriteLine("Press enter to go back.");
228             return selectedPlayer;
229         }
230 
231 
232         //this method will determine the specific spin value once
              generated by the random
233         public static int SpinWheel()
234         {
235             //initialize variables and create an instance of Random()
236             Random rand = new Random();
237             int wheelGenerator;
```

```
238             int[] wheelOfFortune = { 300, 350, 400, 500, 750, 900,
                  1000, 1500, 1750, 2000, 2250, 2500, 2750, 3000, 3250,
                  3500, 3700, 4000, 4250, 4500, 4750, 5000 , -2000, -3000 };
239             wheelGenerator = rand.Next(0, wheelOfFortune.Length);
240
241             //simple animation to display the current spinvalue
242             for (int i = 0; i < 12; i++)
243             {
244                 int displayGenerator = rand.Next(0,
                      wheelOfFortune.Length);
245
246                 Console.Write($"\rSpinning... {wheelOfFortune
                      [displayGenerator],6}");
247                 Thread.Sleep(100 + (i * 20));
248             }
249
250             //this output provides the accurate value of the spin.
251             Console.Write($"\rSpinning... {wheelOfFortune
                  [wheelGenerator],6}");
252             Thread.Sleep(500);
253             Console.WriteLine();
254             return wheelOfFortune[wheelGenerator];
255         }
256
257         //method to return a string value based on the randomized
              string
258         public static string WordRandomizer()
259         {
260             Random rand = new Random();
261             int wordRandomizer;
262             string[] words = { "computerized", "electricity",
                  "chrysanthemum", "programming", "mathematics",
                  "photography", "dictionary", "restaurant"};
263             wordRandomizer = rand.Next(0, words.Length);
264             return words[wordRandomizer];
265         }
266
267         //a method to check if userInput is a letter from the alphabet
268         public static string ValidateUserInput(string userInput,
              char[] alphabet)
269         {
270             bool letterFound = false;
271             //asks the user to keep entering a string
272             do
273             {
274                 //userInput is greated than 1 or no userInput at all
275                 if (userInput.Length != 1)
276                 {
277                     Console.WriteLine("Invalid input! Try again!");
278                     Console.Write("Enter a letter: ");
279                     userInput = Console.ReadLine();
280                 }
281
```

```
282                 else
283                 {
284                     //checks if userInput is equal to the specific
                          letter in the alphabet
285                     for (int i = 0; i < alphabet.Length; i++)
286                     {
287                         if (char.Parse(userInput.ToUpper()) == alphabet
                              [i])
288                         {
289                             letterFound = true;
290                         }
291                     }
292                     //condition if userInput is not a letter
293                     if(letterFound == false)
294                     {
295                         Console.WriteLine("Invalid input! Try again!");
296                         Console.Write("Enter a letter: ");
297                         userInput = Console.ReadLine();
298                     }
299                 }
300             } while (letterFound == false);
301             return userInput;
302         }
303 
304         //method to check if a letter is in the word to be guessed.
305         public static int ValidateCorrectLetters(string wordToGuess,
              char[] wordBlanks, string userInput)
306         {
307             //initialize variables
308             bool correctLetter = false;
309             char validator = char.Parse(userInput);
310             int letterOccurences = 0;
311 
312             //for loop to check letter occurences
313             for(int i = 0; i < wordBlanks.Length; i++)
314             {
315                 if(char.ToLower(wordToGuess[i]) == char.ToLower
                      (validator))
316                 {
317                     correctLetter = true;
318                     wordBlanks[i] = validator;
319                     letterOccurences++;
320                 }
321 
322             }
323 
324             //condition if letter is not in the wordGuess
325             if(correctLetter == false)
326             {
327                 Console.WriteLine("Letter not found!");
328             }
329             else
330             {
```

```
331                 Console.WriteLine("Letter found!");
332             }
333 
334             //returns the number of letter occurences
335             return letterOccurences;
336         }
337 
338 
339         //a method created to validate if userInput is a vowel
340         public static bool IsVowel(string userInput)
341         {
342             userInput = userInput.ToLower();
343             if(userInput == "a"||
344                 userInput == "e"||
345                 userInput == "i"||
346                 userInput == "o"||
347                 userInput == "u")
348             {
349                 return true;
350             }
351             return false;
352         }
353 
354 
355         //the spin wheel method
356         public static Student TaskFive(Student selectedPlayer)
357         {
358             //initialize variables
359             int wheel = 0, currentScore = selectedPlayer.score,
                  letterOccurences = 0;
360             string wordToGuess = "", userInput = "", updatedWord = "";
361             char[] alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ".ToCharArray
                  ();
362 
363             //a constant for a vowel amount
364             const int VOWEL_REDUCE = 250;
365             bool isVowel = false;
366 
367             //get the word from WordRandomizer method
368             wordToGuess = WordRandomizer();
369 
370             //create blanks based on the wordGuess length
371             char[] wordBlanks = new char[wordToGuess.Length];
372             for (int i = 0; i < wordToGuess.Length; i++)
373             {
374                 wordBlanks[i] = '_';
375             }
376 
377             //spin the wheel
378             do
379             {
380                 Console.Clear();
381                 //display the blanks
```

```
382                 for (int i = 0; i < wordBlanks.Length; i++)
383                 {
384                     Console.Write($"{wordBlanks[i]} ");
385                 }
386                 Console.WriteLine();
387                 //display the available letters
388                 for(int i = 0; i < alphabet.Length; i++)
389                 {
390                     Console.Write($"{alphabet[i]} ");
391                 }
392 
393 
394                 Console.WriteLine("\n");
395                 wheel = SpinWheel();
396                 Console.WriteLine("\n\n");
397                 Console.WriteLine($"The wheel landed on: {wheel}");
398 
399                 Console.WriteLine("\n");
400                 Console.WriteLine($"Current score: {currentScore}");
401 
402                 //validate the input if it is a vowel and score is
                      less than 250
403                 do
404                 {
405                     Console.Write("Enter a letter: ");
406                     userInput = Console.ReadLine();
407                     userInput = ValidateUserInput(userInput, alphabet);
408                     isVowel = IsVowel(userInput);
409                     if(isVowel == true && currentScore < VOWEL_REDUCE)
410                     {
411                         Console.WriteLine("Can't afford a vowel yet!");
412                     }
413                 } while (isVowel == true && currentScore <
                      VOWEL_REDUCE);
414 
415                 //removes a letter that was already used
416                 for(int i = 0; i < alphabet.Length; i++)
417                 {
418                     if (alphabet[i] == char.Parse(userInput.ToUpper()))
419                     {
420                         alphabet[i] = ' ';
421                     }
422                 }
423 
424                 //calls the function to see if letters have occured
425                 letterOccurences = ValidateCorrectLetters(wordToGuess,
                      wordBlanks, userInput);
426 
427 
428                 //condition to check the spinvalue
429                 if (wheel == -2000 || wheel == -3000)
430                 {
431                     currentScore += wheel;
```

```
432                 }
433                 else if(isVowel == true)
434                 {
435                     currentScore -= VOWEL_REDUCE;
436                 }
437                 else
438                 {
439                     currentScore += letterOccurences * wheel;
440                 }
441 
442                 //parses wordBlanks to string and assigns to
                      updatedWord
443                 //to display if the word has been solved
444                 updatedWord = new string(wordBlanks);
445                 Console.ReadLine();
446                 //final display
447                 Console.Clear();
448 
449                 //gives the illusion of resetting the used letters, if
                      the problem has been solved
450                 for (int i = 0; i < wordBlanks.Length; i++)
451                 {
452                     Console.Write($"{wordBlanks[i]} ");
453                 }
454                 Console.WriteLine();
455 
456                 for (int i = 0; i < alphabet.Length; i++)
457                 {
458                     Console.Write($"{alphabet[i]} ");
459                 }
460                 Console.WriteLine();
461 
462                 Console.WriteLine($"\nCurrent score: {currentScore}");
463 
464                 //if else condition to check if updatedWord to lower
                      is equal to
465                 //wordToGuess to lower
466                 if (updatedWord.ToLower() == wordToGuess.ToLower())
467                 {
468                     Console.WriteLine("You have guessed the word!");
469                     Console.WriteLine($"The word is: {wordToGuess}");
470                     Console.WriteLine("Press enter to view the top 5");
471                     Console.ReadLine();
472                 }
473 
474                 //spins the wheel again
475                 else
476                 {
477                     Console.WriteLine("Press enter to spin again!");
478                     Console.ReadLine();
479                 }
480             } while (updatedWord.ToLower() != wordToGuess.ToLower());
481 
```

```
482             //sets the current score to the player
483             selectedPlayer.score = currentScore;
484             return selectedPlayer;
485         }
486 
487 
488         //method to update the text file
489         public static void UpdateScore(Student[] student,Student
              selectedPlayer)
490         {
491             sw = new StreamWriter(@"../../../wheelOfFortune.txt");
492             //update current score
493             for(int i = 0; i < student.Length; i++)
494             {
495                 if (student[i].firstName == selectedPlayer.
                      firstName && student[i].lastName ==
                      selectedPlayer.lastName)
496                 {
497                     student[i].score = selectedPlayer.score;
498                 }
499             }
500 
501             //write in text file
502             for(int i = 0; i < student.Length; i++)
503             {
504                 sw.WriteLine(student[i].firstName);
505                 sw.WriteLine(student[i].lastName);
506                 sw.WriteLine(student[i].interest);
507                 sw.WriteLine(student[i].score);
508             }
509             sw.Close();
510         }
511 
512 
513         //displays the top five players
514         public static void TopFive(Student[] student)
515         {
516             //used bubble sort to order players according to their
                  scores
517             for(int i = 0; i < student.Length; i++)
518             {
519                 for(int j = 0; j < i; j++)
520                 {
521                     if (student[i].score > student[j].score)
522                     {
523                         Student temp = student[i];
524                         student[i] = student[j];
525                         student[j] = temp;
526                     }
527                 }
528             }
529 
530             //neatly output top 5 players
```

```
531             Console.WriteLine("Top 5:\n");
532             Console.Write("First Name".PadRight(25));
533             Console.Write("Last Name".PadRight(25));
534             Console.Write("Interest".PadRight(50));
535             Console.WriteLine("Score".PadRight(25));
536             Console.WriteLine();
537             for (int i = 0; i < 5; i++)
538             {
539                 Console.Write($"{student[i].firstName}".PadRight(25));
540                 Console.Write($"{student[i].lastName}".PadRight(25));
541                 Console.Write($"{student[i].interest}".PadRight(50));
542                 Console.WriteLine($"{student[i].score}".PadRight(25));
543             }
544             Console.WriteLine("\n\nPress enter to go back to main
                  menu.");
545         }
546
547         //method to exit program
548         public static void TaskZero()
549         {
550             Console.WriteLine("Exiting program. Thank you and have a
                  good day!\n");
551         }
552
553         //error message in the main menu
554         public static void ErrorMessage()
555         {
556             Console.WriteLine("Wrong input! Please try again!\n");
557         }
558
559
560         //main menu UI
561         public static void TaskChoices()
562         {
563             Console.Clear();
564             Console.SetCursorPosition(0, 0);
565             Console.WriteLine("1 - List all contestants");
566             Console.WriteLine("2 - Locate a player");
567             Console.WriteLine("3 - Generate 10 finalists");
568             Console.WriteLine("4 - Select the player from the
                  finalists");
569             Console.WriteLine("5 - Play the game");
570             Console.WriteLine("0 - Exit the game");
571             Console.Write("Please Enter a task number: ");
572             userChoice = int.Parse(Console.ReadLine());
573             Console.Clear();
574             Console.SetCursorPosition(0, 0);
575         }
576
577         //main menu method that accepts struct and returns the
              selected student
578         public static Student Output(Student[] student, Student[]
              finalists, Student selectedPlayer)
```

```
579         {
580             //display main menu UI
581             TaskChoices();
582 
583             //selects a specific task
584             switch (userChoice)
585             {
586                 case 1:
587                     //display all students
588                     TaskOne(student, selectedPlayer);
589                     Console.WriteLine();
590                     break;
591                 case 2:
592                     //select a specific student
593                     TaskTwo(student);
594                     break;
595                 case 3:
596                     //generate finalists
597                     TaskThree(student, finalists);
598                     break;
599                 case 4:
600                     //generate a player
601                     //condition if a finalists is null
602                     if (string.IsNullOrWhiteSpace(finalists
                          [0].firstName))
603                     {
604                         Console.WriteLine("You need to generate
                              finalists first!");
605                         Console.WriteLine("Press enter to go back.");
606                     }
607                     else
608                     {
609                         selectedPlayer = TaskFour(finalists,
                              selectedPlayer);
610                     }
611                     break;
612                 case 5:
613                     //play the game
614                     //condition if selectedPlayer is null
615                     if(string.IsNullOrWhiteSpace
                          (selectedPlayer.lastName))
616                     {
617                         Console.WriteLine("You need to find a player
                              first!");
618                         Console.WriteLine("Press enter to go back.");
619                     }
620                     else
621                     {
622                         //selected player plays the game
623                         selectedPlayer = TaskFive(selectedPlayer);
624 
625                         //updates the score and writes it in the text
                              file
```

```
626                         UpdateScore(student, selectedPlayer);
627                         Console.Clear();
628
629                         //displays top 5 as soon as the game is
                              finished
630                         TopFive(student);
631                     }
632                     break;
633                 case 0:
634                     //exits the system
635                     TaskZero();
636                     break;
637                 default:
638                     //userinput is wrong
639                     ErrorMessage();
640                     break;
641             }
642             Console.ReadLine();
643             return selectedPlayer;
644         }
645     }
646 }
647
```